

首次拜访 H 公司负责 IT 服务的总经理，胡总。

公司包括好几个部门和子公司，主要做电力相关的软件项目，也服务集团内信息化需求。胡总说希望用 12 个月时间改善开发项目的质量。

我简单介绍了很多开发团队都因为前面工作没有关注质量，导致大量缺陷在后期才暴露，除了影响质量，也导致大量返工。他们赞同这总体改进方向后，我继续说若要有效果，必须跟做项目一样要策划与监控。但很多团队因没有做好 WBS（工作任务分解），无法做好计划和监控开发过程中的偏差（进度、工作量）。

胡总说：“上周刚与甲方开会，问我们项目的进展，偏差多少，因缺乏相关项目信息，说不出来、所以项目组长须要了解项目管理的步骤，每步做什么？有什么产出？”

我：“赞同。项目经理不仅要制定项目计划和进度表，也要依据实际任务的完成情况监控，汇报项目的偏差，例如延误了 15%”

总监说：“还要明确 15% 是什么意思。”

我：“所以建议项目管理都使用挣值分析计算偏差百分比，避免误解。必须有机制知道任务实际是否完成，才能监控项目实际的进展情况。例如：任务的产出物必须通过同行评审才算是 100% 完成。

“过程改进的项目计划应该如何制定？有什么步骤？”技术研究中心主任问。

（中心负责指导和辅助各开发部门成立改进小组推进过程改进）

“步骤跟软件开发项目类似，也是包括：

先了解需求、项目的目标、成功要素，立项

按项目和团队特性选择项目生命周期

工作任务分解和规模估算

成本预算、时间安排、分配资源

按里程碑监控

但因为需求比较明确，规模固定，有些地方可以简化。”

总监说：“你就跟老师学习一下，学好了可以后面辅导项目团队，如何做好估算、策划和监控，解决当前无法提到项目偏差这难题。”

会议结束后，我便与主任继续讲解如何开展，制定计划。

=====

与软件开发项目不同，今次过程改进项目的需求和目标很明确。

但还需要跟所有干系人开会确认，并写出带出项目概述（一页纸内），

里面包括：

* 目标、预计结果、度量、里程碑、主要活动

* 人员：组长和组员

因为你们第一次做，不熟识，这模板已经把已经知道的内容填上，你填空便可，但有些地方需要你按实际项目情况写。（详见附件）

成功准则非常重要，因最终必须能为公司省钱，所以概览里明确会预计 3 个月剩余多少钱（或返工工作量）；

后期暴露的缺陷数会减多少。

制定具备详细计划前要选择项目生命周期

要考虑的因素包括

* 需求是否明确？

* 过程中会不会有很多需求变更？

* 是否会探索很多新的技术？或者已经很成熟，做过同类项目很多次？

* 团队成员的经验和能力

“正如你所说，需求很明确，出现需求变更的可能性低。

但我们没有做过类似的项目，团队成员也没有相关经验。”主任说。

赞同。所以就不合适选择传统的瀑布式生命周期：

前期花很多精力做好需求，写需求/设计文档。

因为大家都没做过，也无法写出合适的过程文档和指南，

应该用有限的人力资源投放在试点：

学习如何收集数据；分析根因；提出对应纠正措施；并马上在下一代尝试

所以应选择迭代生命周期

如果对应的软件开发项目也是按迭代生命周期，每 2-4 周一个迭代，

两者就可以同步了。

如果需求和技术方面都存在很多不确定的变数，例如研发全新，前所未有的新产品，

而且团队成员能力高强，

如一些全新的研发项目：

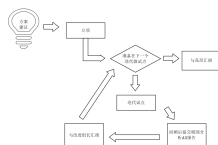
使相信团队成员有能力在每轮冲刺，开发出优秀产品为公司增值。

（减少传统的活动策划和指标监控项目里程碑）

和前面谷歌创始人 Larry 先生对芬兰计划的评价一样，传统计划方式反而会妨碍了员工的创新。

虽然有很多不同类型迭代生命周期，我们可以简单按原型法迭代的方式。可以把每一轮试点改进当成做一轮原型，方便理解。

迭代项目管理



因为需求较明确：针对如何能让缺陷预先发现解决，减少大量后期返工
建议类似原型法，使用迭代项目生命周期：

* 立项 (如收集需求)

* 准备在下一个迭代做试点 (如策划下一个原型)

* 迭代试点 (如开发原型)

* 回顾后提交根因分析 A3 报告 (如交付原型)

* 与改进组长汇报 (除了 A3 报告，也包括项目监控) (如收集客户反馈)

* 里程碑与高层汇报 (与计划对比) (如最终交付)

工作任务分解 WBS

可以按这种迭代方式做工作任务分解。

例如

以上试点的迭代活动，再加上培训，汇报，开会，写经验教训 wiki
WBS 的第一层应包括：

- * 迭代准备与策划
- * 迭代里的准备（包括收集数据）
- * 一起做迭代回顾，分析根因
- * 写每轮迭代 A3 报告，更新缺陷和返工趋势图
- * 参加会议，与高层、EPG 汇报
- * 培训
- * 在 Wiki 写经验教训、过程、指南等

因为项目不大，活动应该都不会超过五人天，不用再细分

定了 WBS 以后，需要记录在项目管理工具里，
每个活动除了开始结束日期外，都应该有明确的人员和产出物。

每次迭代汇报就是一个里程碑。

假如你们四个分公司的改进目标需求都是一样，你们的项目规模应该是一样，
所以就不需要像软件开发项目按输出输入多少估算功能点数。

但每个小组还是可以依据他们以前的同类项目数据（历史数据），更准确估算下
轮迭代某活动，例如在 wiki 写迭代 A3 报告，的工作量。

如果除了返工工作量，也想估算返工成本，可以按员工人均均为工时的成本计算，
得出项目的返工成本。

= = = = =

如何监控项目偏差？

很多敏捷教材包括 SCRUM，使用：

- * 燃尽图：按过去的故事点完成趋势预估项目的完成日期，与本来计划完成日期
差多少。但因项目规模大小不同，不能用偏差的天数，例如会晚 15 天完成，代
表项目的偏差率。
- * 使用看板把项目的进度可视化：干系人可从看板可以看到哪些开发活动到了
什么阶段，哪些已经完成，但还是给不了延误百分多少。

有些团队 2-4 周一轮迭代，策划时只是利用故事点估计每一轮迭代能完成多
少模块的开发，没有再细分到开发活动。

因为没有细分到任务 (task) 并估计每条任务需要的人天，很容易出现帕金森
现象，大家都只会看到最终迭代结束日期的节点，导致延误。

“今天才周二，离本冲刺结束日还有 3 周多，今周不如先研究我负责这模块所
需的技术，下周 3 左右开始写也来得及。”开发人员会这样想。

没有考虑任务间的依赖关系，例如：测试用例的执行必须要对应的模块开发
完成后才可以开始；但测试用例须要尽早写好，给开发人员参考。所以要满
足管理层关于项目偏差监控的要求，和减少任务的延迟，需要有 WBS 管理
功能的项目管理工具，配合挣值分析才能有效策划与监控项目。

监控可视化

在传统项目管理除了燃尽图，还有以下方法：

- 1. 甘特图，
- 2. 按里程碑的挣值系数趋势图，
- 3. 红黄绿灯

1. 甘特图，

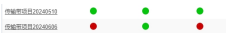
500px

从甘特图看到每个活动的实际开始与结束日期，与计划的偏差。
2. 按每次迭代的挣值系数趋势图，（如果每轮迭代都按计划完成，所有报告也完成，进度偏差应是零。）



从上图看到 C（CPI），和 S（SPI）都是每个迭代越来越差（往下），预算超出，进度落后。

3. 红黄绿灯



以上只是监控项目活动执行情况，有没有偏差。更关键应看每组返工工作量和缺陷数量的趋势，是否有改善。如发现与本来的目标差异很大，便应与高层开会讨论，是否应调整目标与计划。

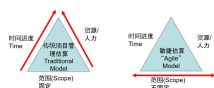
如果 WBS 项目管理工具能支持以上可视化监控功能，如大家都每周五按 WBS 填写工时表，组长不须要画额外工作量，所有人，包括高层等，都可以即时在系统看到以下图表和系数监控项目进展

::= == == ==

我只是用某个小组的计划来解释，较容易理解，但其实你们中心也要制定你们的计划，因为你们有很多前期准备工作，也需要用计划来监控。

如何用于软件开发

总体思路不变，例如：开头还是要与干系人确认出一页纸的项目预览 (POS)。然后按项目需求选择合适的项目生命周期。过程改进项目，因为需求明确，选用迭代原型法便可以。从范围（需求），进度、成本、质量和人力资源等要求，判断这软件开发项目应使用那类项目生命周期。如果范围（需求）明确，不允许修改，进度成本和人力资源便应依据项目需求制定时间和资源安排（瀑布式）。敏捷开发是假定时间与人手固定 (Time Boxing)，但范围可变，团队便应请简策划，在每轮迭代里先开发最有价值的功能，给客户的价值最大化。如下图：



如果团队因需求不确定，选择敏捷开发方式，虽然项目启动后还是要准备 WBS 工作任务分解，但不需要像传统瀑布式把几个月里面的所有活动都细分，减少浪费，只需要先制定第一级分解便可以。功能需求也要须要分优先级，例如：可以用 MoSCoW 分 4 类。也需要定每一轮的时长，例如 2 周，和预计总共要多少轮才能完成。经过与干系人确认审批，便可以开始每一轮的策划。如下图的上半部分：敏捷项目框架 (Agile Project Framework APF) 流程：

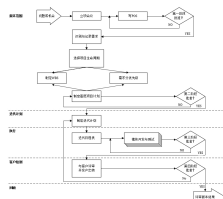


Fig 11-9

每个项目由多轮迭代组成，开头的发版范围除了 POS 外，也应包含：

- 需求分分解 (Requirements Breakdown Structure)，并分优先级
- 总体 WBS 和依赖关系

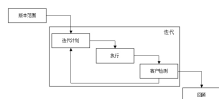


Fig 11-10

与传统瀑布式项目管理不同，每轮迭代只需要做本迭代的 WBS，不会花精力在未来未知的迭代上，满足精益思路，也可以有 WBS 管理的好处，两者取得平衡。

如何管理每轮的活动依赖和人手分配

某敏捷团队希望开发一款新的游戏，每两周一轮迭代，他们按项目所需要的需求（故事点），制定了项目的总体发布计划，第一周，团队成员包括开发与测试人员，针对某一功能一起头脑风暴，按照第一轮迭代的故事估算工作量，如下图：

	活动	人时
A	状态管理类编程	16
B	为 A 做自动化测试	10
C	让机器人追踪没有阻挡的环	12
D	为 C 做自动化测试	12
E	虽然玩家试图阻挡，但机器人还是继续建环	8
F	为 E 写测试用例	4
G	为 E 做自动化测试	4
H	让机器人阻挡玩家建环	12
J	为 H 写测试用例	4
K	为 H 做自动化测试	2

因为估计总共需要 84 人时，按每工作日每人有 6 小时工作，（减出沟通开会等时间），也考虑团队成员有哪天会请假或者被其他项目占用时间，然后直接问这三位成员是否可以。

你觉得以上的敏捷项目管理方法（参考 Cohn 先生在他书中的案例）有什么不足？

“好像没有考虑活动间的依赖关系，也不知道自己负责的活动从哪一天开始，哪一天结束？”

如何用 **APF** 方法完善

先把活动工作量的颗粒度从小时改成每半天：

	活动	人时	
A	状态管理类编程	16	2
B	为 A 做自动化测试	10	1.5
C	让机器人追踪没有阻挡的环	12	1.5
D	为 C 做自动化测试	12	1.5
E	虽然玩家试图阻挡，但机器人还是继续建环	8	1
F	为 E 写测试用例	4	0.5
G	为 E 做自动化测试	4	0.5
H	让机器人阻挡玩家建环	12	1.5
J	为 H 写测试用例	4	0.5
K	为 H 做自动化测试	2	0.5

使用 WBS 项目管理把依赖关系，每个活动的开始日期，结束日期再加上人员安排，例如：可以在墙上的白板或大白纸，用便利贴、水笔画上本迭代里活动间的依赖关系，在下面按每轮 2 周，填活动安排，包括每活动上三名开发或测试人员资源的，如下图：

400px

不需要用软件项目管理工具，便可以做到两周迭代的详细计划。在两周的人员安排表只需要分到半天不需要分到小时，每天早上定期十五分钟站立会议，每人汇报实际的进展情况，并更新白板上的计划。可以最简单低科技做到每轮策划与监控的作用，大家一看白板上的图就知道现在项目今次迭代的当前状态。能否在两周内完成承诺的组件。

因为每轮迭代只有两到四周，用大白纸和便利贴，这些低科技方法便可以有效策划和监控，改善之前敏捷策划方法的不足，包括：

- 每活动有开始和结束日期
- 活动间的依赖关系
- 人手分配

也尽量减轻管理工作的负担。

迭代结束并与客户展示确认后，可以按迭代完成情况算出当前的 EV 挣值 AC 实际成本 PV 计划，例如：如果计划里的所有模块都已经完成，客户也接受，也没有增加其他的模块开发，EV 便等于 PV，进度没偏差。

如果要全面了解各类项目管理方法与技巧，从需求到结项，便要参考项目管理的书籍。但必须挑选针对软件开发项目，并包括敏捷开发的内容。

刚才跟你介绍的步骤，都能在《有效项目管理》这本书找到相关内容。你们项目组也可以用这次机会，开始养成定计划，然后按里程碑监控的好习惯。后面可以使用同样步骤，加上功能点规模估算，制定开发项目的 WBS 计划。

总结

你自己可以首先使用刚才的原型法制定公司总体的过程改进计划（要注意刚才我举的例子只是改进小组的计划，并不是公司总体计划），并要求项目组针对改进目标下制定自己的改进项目计划，他们后面可以再使用 APF 的方法，针对软件开发项目制定计划和监控，后面便应可以回应开会时胡总要实时监控项目偏差的要求。

附件

项目一页概览 POS

机会或问题描述

- 由于缺陷没有得到适当的审查或测试而造成的长期浪费
- 质量问题：顾客拒收，顾客不满，影响 H 品牌形象
- 由于资源大部分时间用于“技术债务”（返工以纠正缺陷）而导致的延迟

目标

- 通过在早期阶段提高缺陷去除效率，例如评审和测试
- 结果和时间框架：
 - * 将迭代中的系统测试缺陷密度从当前（2024 年 10 月）的 xx/FP 提高到 2025 年 1 月的 yy/FP
 - * 将每次迭代的总返工工时从当前（2024 年 10 月）的 xx 人-hr/FP 减少到 2025 年 1 月的 yy 人-hr/FP
- 示例：
 - * 经过问题诊断和补救措施，将导致 ST 缺陷在实施后 3 个月内减少 50%。初步计算估计节省的努力（成本）潜力约为。总项目工作日的 30%（由于在需求和开发阶段通过早期移除缺陷减少了返工作量）

预计结果/度量/时段/行动

- 针对试点项目，培训团队如何做好迭代回顾分析缺陷（Eva 11 月 15 日）

- 从 JIRA 收集迭代缺陷数
- 返工工作将在 sprint 回顾中手动收集（在 Nov 培训里跟团队说好大家应如何收集返工工作量）
- 目标：回顾时一起分析根因（有 ATM 现场辅导），在下一轮迭代使用纠正措施，希望能把迭代系统测试缺陷数减半

关键里程碑

1. 将测试重点放在容易出错的模块上（10 月 10 日/ Peter）
2. 设计测试用例培训（11 月 10 日/ John）

注意：详细的时间表请参考 Jira

=== 成功标准 === “我们为什么要做这个项目？”，衡量这个项目的业务结果
“我们（和我们的客户）必须做些什么才能说项目是成功的？”

* 由于“技术债务”导致的返工工作显著减少

* 在 UAT 和用户开始使用产品后的前两个月发现的缺陷显著减少（注）

注：参考过程改进项目建议可行性研究中的缺陷返工和 ROI 估计中的数字

项目经理与组员

- 组长：陈乔
- 成员：李明明, 华志, 胡天

参考 References

1. Wysocki, R. *Effective Project Management* 6/e (2012)
2. Cohn, Mike, *Agile Estimating and Planning* (2006)